# SMART DIGITAL EXPENSE TRACKER

Lab Sheet–09 | BCA I Semester | Session 2026–2027

## 1. Objective

To develop a smart expense management system in C++ that records daily expenses, categorizes spending, monitors a monthly budget, generates reports, and predicts whether the budget may be exceeded.

## 2. Algorithm

1. Start the program and load saved expense records.
2. Display the menu and accept the user's choice.
3. Add, view, search, delete, or categorize expenses as selected.
4. Set a monthly budget and calculate total spending.
5. Estimate monthly spending from the average daily expense.
6. Display a warning if the predicted expense exceeds the budget.
7. Save records to a file and exit.

## 3. C++ Source Code

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>
using namespace std;

struct Expense {
    int id;
    string date, category, description;
    double amount;
};

Expense expenses[1000];
int count = 0;
double budget = 0;

// Save all expenses to a text file
void saveData() {
    ofstream file("expenses.txt");
    for (int i = 0; i < count; i++) {
        file << expenses[i].id << "|"
             << expenses[i].date << "|"
             << expenses[i].category << "|"
             << expenses[i].description << "|"
             << expenses[i].amount << endl;
    }
    file.close();
}

// Load previously saved expenses
void loadData() {
    ifstream file("expenses.txt");
    string line;

    while (getline(file, line) && count < 1000) {
        size_t p1 = line.find('|');
        size_t p2 = line.find('|', p1 + 1);
        size_t p3 = line.find('|', p2 + 1);
        size_t p4 = line.find('|', p3 + 1);

        if (p1 == string::npos || p2 == string::npos ||
            p3 == string::npos || p4 == string::npos)
            continue;

        expenses[count].id = stoi(line.substr(0, p1));
        expenses[count].date =
            line.substr(p1 + 1, p2 - p1 - 1);
        expenses[count].category =
            line.substr(p2 + 1, p3 - p2 - 1);
        expenses[count].description =
```

```
            line.substr(p3 + 1, p4 - p3 - 1);
        expenses[count].amount = stod(line.substr(p4 + 1));
        count++;
    }
    file.close();
}

// Add a new expense
void addExpense() {
    if (count >= 1000) {
        cout << "Expense limit reached!\\n";
        return;
```

## 3. C++ Source Code (continued)

```
    }

    expenses[count].id = count + 1;
    cout << "Enter date (DD-MM-YYYY): ";
    cin >> expenses[count].date;
    cout << "Enter category: ";
    cin >> expenses[count].category;
    cout << "Enter description: ";
    cin.ignore();
    getline(cin, expenses[count].description);
    cout << "Enter amount: ";
    cin >> expenses[count].amount;

    count++;
    saveData();
    cout << "Expense added successfully!\\n";
}

// Show all recorded expenses
void viewExpenses() {
    if (count == 0) {
        cout << "No expenses found!\\n";
        return;
    }

    cout << "\\n--- Expense History ---\\n";
    for (int i = 0; i < count; i++) {
        cout << "ID: " << expenses[i].id
             << " | Date: " << expenses[i].date
             << " | Category: " << expenses[i].category
             << " | Description: " << expenses[i].description
             << " | Amount: Rs." << expenses[i].amount
             << endl;
    }
}

// Calculate the total amount spent
double totalExpense() {
    double total = 0;
    for (int i = 0; i < count; i++)
        total += expenses[i].amount;
    return total;
}

// Display spending for a selected category
void categoryReport() {
    string category;
    double total = 0;
    cout << "Enter category: ";
    cin >> category;

    for (int i = 0; i < count; i++) {
        if (expenses[i].category == category)
            total += expenses[i].amount;
    }

    cout << "Total " << category
         << " expense = Rs." << total << endl;
}

// Display monthly expense summary
void monthlyReport() {
```

## 3. C++ Source Code (continued)

```
    double total = totalExpense();
    cout << "\\nTotal Expense = Rs." << total << endl;
    cout << "Monthly Budget = Rs." << budget << endl;
    cout << "Remaining Budget = Rs."
         << budget - total << endl;
}

// Set the monthly budget
void setBudget() {
    cout << "Enter monthly budget: ";
    cin >> budget;
    cout << "Budget set successfully!\\n";
}

// Estimate spending and warn about the budget
void predictExpense() {
    double total = totalExpense();
    int days;

    cout << "Enter days passed in month: ";
    cin >> days;

    if (days <= 0 || days > 31) {
        cout << "Invalid number of days!\\n";
        return;
    }

    double predicted = (total / days) * 30;
    cout << fixed << setprecision(2);
    cout << "Current Expense = Rs." << total << endl;
    cout << "Predicted Monthly Expense = Rs."
         << predicted << endl;

    if (budget > 0 && predicted > budget)
        cout << "WARNING: Budget may be exceeded!\\n";
    else
        cout << "Your spending is within budget.\\n";
}

// Search for an expense by its ID
void searchExpense() {
    int id;
    cout << "Enter expense ID: ";
    cin >> id;

    for (int i = 0; i < count; i++) {
        if (expenses[i].id == id) {
            cout << "Expense Found!\\n";
            cout << "Date: " << expenses[i].date
                 << "\\nCategory: " << expenses[i].category
                 << "\\nAmount: Rs." << expenses[i].amount
                 << endl;
            return;
        }
    }
    cout << "Expense not found!\\n";
}

// Delete an expense by its ID
void deleteExpense() {
    int id;
    cout << "Enter expense ID: ";
```

## 3. C++ Source Code (continued)

```
    cin >> id;

    for (int i = 0; i < count; i++) {
        if (expenses[i].id == id) {
            for (int j = i; j < count - 1; j++)
                expenses[j] = expenses[j + 1];

            count--;
            saveData();
            cout << "Expense deleted successfully!\\n";
            return;
        }
    }
    cout << "Expense not found!\\n";
}

int main() {
    loadData();
    int choice;

    do {
        cout << "\\n===== SMART EXPENSE TRACKER =====\\n";
        cout << "1. Add Expense\\n";
        cout << "2. View Expense History\\n";
        cout << "3. Category-wise Report\\n";
        cout << "4. Monthly Report\\n";
        cout << "5. Set Budget\\n";
        cout << "6. Budget Prediction\\n";
        cout << "7. Search Expense\\n";
        cout << "8. Delete Expense\\n";
        cout << "9. Exit\\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: addExpense(); break;
            case 2: viewExpenses(); break;
            case 3: categoryReport(); break;
            case 4: monthlyReport(); break;
            case 5: setBudget(); break;
            case 6: predictExpense(); break;
            case 7: searchExpense(); break;
            case 8: deleteExpense(); break;
            case 9:
                saveData();
                cout << "Thank you!\\n";
                break;
            default:
                cout << "Invalid choice!\\n";
        }
    } while (choice != 9);

    return 0;
}
```

## 4. Sample Input and Output

### Adding an expense

```
===== SMART EXPENSE TRACKER =====
Enter your choice: 1
Enter date (DD-MM-YYYY): 03-11-2026
Enter category: Food
Enter description: Lunch
Enter amount: 100
Expense added successfully!
```

### Budget prediction

```
Enter monthly budget: 5000
Enter days passed in month: 10
Current Expense = Rs.2000.00
Predicted Monthly Expense = Rs.6000.00
WARNING: Budget may be exceeded!
```

## 5. Conclusion

The Smart Digital Expense Tracker is a useful C++ project for recording daily expenses, calculating total spending, monitoring a monthly budget, and estimating future expenses. It demonstrates structures, arrays, functions, loops, searching, and file handling.

Note: This is a basic educational implementation. The prediction uses the average daily spending multiplied by 30. The monthly budget is entered during the current program session.